Neural Networks, Approximation Theory, and Autoencoders

ML & Deep Learning Course

Pedro Watuhã dos Santos Guerra

April 4, 2026

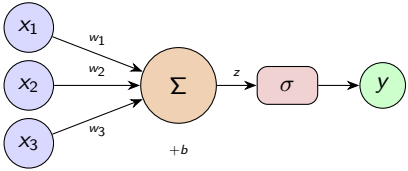
Outline

1. Perceptrons
2. Activation Functions
3. Feedforward Neural Networks
4. Backpropagation
5. Approximation Theory
6. The Neural Network Zoo
7. The Manifold Hypothesis
8. Autoencoders and PCA

The Biological Neuron and Its Abstraction

Biological neuron (simplified):

- **Dendrites**: receive input signals from other neurons
- **Soma**: integrates (sums) incoming signals
- **Axon**: fires if total signal exceeds threshold
- **Synapse**: connection strength (weight) between neurons



$$z = \sum_{j=1}^{p} w_j x_j + b, \qquad y = \sigma(z)$$

McCulloch–Pitts Neuron (1943)

First mathematical model: binary inputs $x_j \in \{0, 1\}$, weights w_j, threshold θ:

$$y = \mathbf{1}\left[\sum_j w_j x_j \geq \theta\right]$$

Rosenblatt's Perceptron (1958)

Binary Classification

Given data $(x_i, y_i) \in \mathbb{R}^p \times \{-1, +1\}$, the perceptron predicts:

$$\hat{y}_i = \mathrm{sign}(\mathbf{w}^\top x_i + b)$$

Learning rule: For each misclassified example:

$$\mathbf{w} \leftarrow \mathbf{w} + \eta\, y_i\, x_i, \qquad b \leftarrow b + \eta\, y_i$$

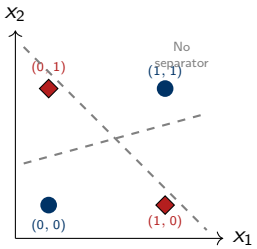
Theorem 1 (Perceptron Convergence)

If data is linearly separable with margin $\gamma > 0$ and $\|x_i\| \leq R$, the perceptron converges in at most

$$T \leq \left(\frac{R}{\gamma}\right)^2$$

The XOR Problem (Minsky & Papert 1969)

XOR is not linearly separable — no hyperplane can classify it.

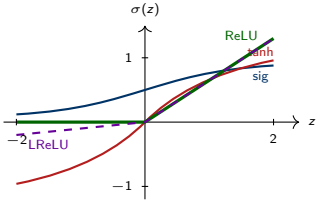


Activation Functions: From Threshold to Smooth

Requirements for a good activation

1. Non-linear (otherwise NN collapses to linear model)
2. Differentiable (for backpropagation)
3. Non-saturating (avoids vanishing gradients)
4. Computationally cheap

Activation	Formula	Saturates?
Sigmoid	$1/(1+e^{-z})$	Both sides
Tanh	$(e^z - e^{-z})/(e^z + e^{-z})$	Both sides
ReLU	$\max(0, z)$	Left only
Leaky ReLU	$\max(\alpha z, z)$	No
GELU	$z\Phi(z)$	Approx. no



Dying ReLU

For $z < 0$: ReLU output and gradient are **both zero**. Neurons receiving mostly negative inputs stop learning permanently.
Fix: Leaky ReLU, GELU, or careful initialization.

Formal Definition: Feedforward Neural Network

Multi-Layer Perceptron (MLP)

A depth-L network with widths $n_0 = p, n_1, \ldots, n_L$ is defined by the recursion:

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}, \qquad \mathbf{a}^{(l)} = \sigma(\mathbf{z}^{(l)}), \quad l = 1, \ldots, L-1$$

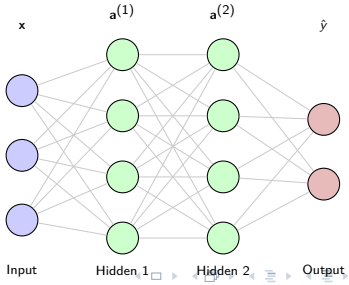
with $\mathbf{a}^{(0)} = \mathbf{x}$ (input) and output $\hat{y} = \mathbf{W}^{(L)}\mathbf{a}^{(L-1)} + \mathbf{b}^{(L)}$.

Parameter count:

$$|\theta| = \sum_{l=1}^{L} n_{l-1} \cdot n_l + n_l = \sum_{l=1}^{L} n_l(n_{l-1} + 1)$$

Example: $p = 784$, hidden layers $512, 256, 128$, output 10:

$$785 \times 512 + 513 \times 256 + 257 \times 128 + 129 \times 10 = 569{,}994$$



Depth and Non-linearity: Why Hidden Layers Help

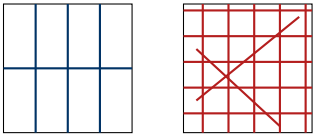
Linear networks collapse

Without nonlinear activations, any L-layer network is equivalent to a single layer:

$$\mathbf{W}^{(L)} \cdots \mathbf{W}^{(1)}\mathbf{x} = \widetilde{\mathbf{W}}\mathbf{x}$$

Depth without nonlinearity adds *no expressivity*.

Depth 1, width 4 Depth 3, width 4



8 regions $\gg$ regions

ReLU creates linear regions

Montufar et al. (2014): A depth-L ReLU network with n units per layer creates up to linear regions

$$\Omega\left(\left(\frac{n}{p}\right)^{p(L-1)} n^p\right)$$

Practical implication

Deeper networks achieve richer function approximation with fewer total parameters than shallow (single hidden layer) networks.

The Learning Problem and Forward Pass

Empirical Risk Minimization

Given data $\{(x_i, y_i)\}_{i=1}^n$, find parameters θ minimizing:

$$\mathcal{L}(\theta) = \frac{1}{n} \sum_{i=1}^{n} \ell(y_i, \hat{y}_i(\theta))$$

Common losses:

- MSE: $\ell(y, \hat{y}) = (y - \hat{y})^2$
- Cross-entropy: $\ell(y, \hat{y}) = -\sum_k y_k \log \hat{y}_k$

Forward Pass: notation

For a single example $\mathbf{x}$, layer by layer:

$$\begin{aligned}
\mathbf{a}^{(0)} &= \mathbf{x} \\
\mathbf{z}^{(l)} &= \mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)} \\
\mathbf{a}^{(l)} &= \sigma(\mathbf{z}^{(l)}), \quad l = 1, \ldots, L-1 \\
\hat{y} &= \mathbf{z}^{(L)} = \mathbf{W}^{(L)} \mathbf{a}^{(L-1)} + \mathbf{b}^{(L)}
\end{aligned}$$

Store all $\mathbf{z}^{(l)}$ and $\mathbf{a}^{(l)}$ — needed for the backward pass.

Stochastic Gradient Descent

$$\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}_{\mathcal{B}}(\theta)$$

where $\mathcal{B}$ is a mini-batch. Requires computing $\nabla_\theta \mathcal{L}$.

The Optimization Problem: Why Backpropagation?

Failure 1: No Analytical Solution

Unlike Ordinary Least Squares ($\hat{\beta} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$), deep networks are built on cascading non-linear activations (like ReLU). The resulting loss landscape is highly non-convex, meaning no closed-form algebraic solution exists.

Failure 2: Numerical Differentiation

Why not just estimate gradients by "tweaking" weights?

$$\frac{\partial \mathcal{L}}{\partial w_i} \approx \frac{\mathcal{L}(w_i + \epsilon) - \mathcal{L}(w_i)}{\epsilon}$$

For a network with P parameters, computing the full gradient requires $P + 1$ **forward passes**.

The Solution: Reverse-Mode Autodiff

Backpropagation solves this by treating the network as a computational graph and applying the *Chain Rule* systematically.

The Dynamic Programming Miracle

Instead of calculating the path from every parameter to the output independently.

- It calculates the error at the output first.
- It passes (caches) this error backward layer by layer.
- **The Result:** It computes *all* partial derivatives simultaneously.

Backpropagation: The Dynamic Programming Shift

The Naive Calculus (Inefficient)

To find the gradient $\nabla_{\mathbf{W}^{(l)}}\mathcal{L}$, the raw chain rule expands into a massive product of Jacobians from the output L all the way down to l:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}} = \underbrace{\frac{\partial \mathcal{L}}{\partial \mathbf{a}^{(L)}} \frac{\partial \mathbf{a}^{(L)}}{\partial \mathbf{z}^{(L)}} \cdots \frac{\partial \mathbf{a}^{(l)}}{\partial \mathbf{z}^{(l)}}}_{\text{Recomputed for every layer!}} \frac{\partial \mathbf{z}^{(l)}}{\partial \mathbf{W}^{(l)}}$$

If we calculate this raw chain independently for every layer, we waste millions of operations recalculating the exact same upstream paths. It scales at $\mathcal{O}(L^2)$.

The Backprop Insight (Efficient)

Instead of looking from the parameter forward, we look from the error backward. We define a single, reusable "error signal" for the pre-activation hub of each layer:

$$\boldsymbol{\delta}^{(l)} \equiv \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(l)}}$$

If we can compute $\boldsymbol{\delta}^{(l)}$ efficiently, the weight update is simply the error multiplied by the input that caused it:

$$\nabla_{\mathbf{W}^{(l)}}\mathcal{L} = \boldsymbol{\delta}^{(l)} \left(\mathbf{a}^{(l-1)}\right)^{\top}$$

The Goal: Find a fast, recursive formula for $\boldsymbol{\delta}^{(l)}$.

Backpropagation: Deriving the Recursive Step

How Error Flows Backward

To find $\boldsymbol{\delta}^{(l)}$, we must pull the known error from the layer above ($\boldsymbol{\delta}^{(l+1)}$) backward through the forward-pass operations: $\mathbf{z}^{(l)} \xrightarrow{\sigma} \mathbf{a}^{(l)} \xrightarrow{\mathbf{W}^{(l+1)}} \mathbf{z}^{(l+1)}$.

Step 1: Through the Weights

First, how does the activation $\mathbf{a}^{(l)}$ affect the loss? It passes through the weight matrix $\mathbf{W}^{(l+1)}$.

$$\frac{\partial \mathcal{L}}{\partial \mathbf{a}^{(l)}} = \left(\mathbf{W}^{(l+1)}\right)^{\top} \underbrace{\frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(l+1)}}}_{\boldsymbol{\delta}^{(l+1)}}$$

Step 2: Through the Activation

Next, how does the raw hub $\mathbf{z}^{(l)}$ affect $\mathbf{a}^{(l)}$? Through the activation function σ.

$$\boldsymbol{\delta}^{(l)} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}^{(l)}} \odot \sigma'(\mathbf{z}^{(l)})$$

Backpropagation: Algorithm and Vanishing Gradients

Full Algorithm (single example)

Forward: compute $\mathbf{z}^{(l)}, \mathbf{a}^{(l)}$ for $l = 1, \ldots, L$
Compute loss $\mathcal{L} = \ell(y, \hat{y})$
$\boldsymbol{\delta}^{(L)} \leftarrow \nabla_{\hat{y}} \ell$
for $l = L - 1$ downto 1 **do**
    $\boldsymbol{\delta}^{(l)} \leftarrow \left(\mathbf{W}^{(l+1)}\right)^\top \boldsymbol{\delta}^{(l+1)} \odot \sigma'(\mathbf{z}^{(l)})$
end for
for $l = 1$ to L **do**
    $\nabla_{\mathbf{W}^{(l)}} \mathcal{L} \leftarrow \boldsymbol{\delta}^{(l)} (\mathbf{a}^{(l-1)})^\top$
    $\nabla_{\mathbf{b}^{(l)}} \mathcal{L} \leftarrow \boldsymbol{\delta}^{(l)}$
end for

Rumelhart, Hinton & Williams (1986) popularized this.
The chain rule derivation dates to Werbos (1974).

Vanishing Gradients

For sigmoid: $\sigma'(z) = \sigma(z)(1 - \sigma(z)) \leq \frac{1}{4}$.
After L layers:

$$\left\|\boldsymbol{\delta}^{(1)}\right\| \sim \left(\frac{1}{4}\right)^L \left\|\boldsymbol{\delta}^{(L)}\right\| \xrightarrow{L \to \infty} 0$$

Early layers receive essentially **no gradient signal**.

Universal Approximation Theorem

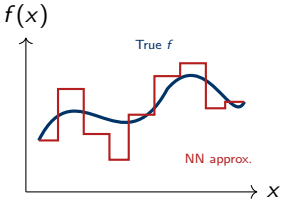
Theorem 2 (Cybenko 1989; Hornik, Stinchcombe & White 1989)

Let $\sigma : \mathbb{R} \to \mathbb{R}$ be any continuous, bounded, non-constant function (e.g. sigmoid). For any continuous $f : [0,1]^d \to \mathbb{R}$ and $\varepsilon > 0$, there exist $N \in \mathbb{N}$, weights $w_j \in \mathbb{R}^d$, biases $b_j \in \mathbb{R}$, and output weights $c_j \in \mathbb{R}$ such that:

$$\sup_{\mathbf{x} \in [0,1]^d} \left| f(\mathbf{x}) - \sum_{j=1}^{N} c_j \, \sigma(w_j^\top \mathbf{x} + b_j) \right| < \varepsilon$$

What UAT does *not* say

- It is an **existence** result — does not say how large N must be
- It does **not** provide a learning algorithm
- It does **not** guarantee generalization
- Requires a **single** hidden layer to be ∞-wide in the worst case



Barron's Theorem: Quantitative Rates

Barron (1993) — L^2 Approximation Rate

Define the *spectral norm* of f:

$$C_f = \int_{\mathbb{R}^d} \|\omega\| \left|\hat{f}(\omega)\right| d\omega, \qquad \hat{f}(\omega) = \int_{\mathbb{R}^d} e^{i\omega^\top x} f(x)\, dx$$

If $C_f < \infty$, then for any distribution μ on a bounded domain and any $N \geq 1$:

$$\inf_{f_N \in \mathcal{F}_N} \int \left(f(\mathbf{x}) - f_N(\mathbf{x})\right)^2 d\mu(\mathbf{x}) \;\leq\; \frac{C_f^2}{N}$$

where $\mathcal{F}_N$ is the class of single-hidden-layer NNs with N units and sigmoidal σ.

Why this is remarkable

- Rate $O(1/N)$ is **dimension-free**
- Compare: k-th order polynomial approximation in $\mathbb{R}^d$ requires $O(N^{2k/d})$ terms for the same error.
- Barron's class of functions "sidesteps" the curse of dimensionality

Benign Overfitting in NNs: The Linear Regime

The Tractability of Overparametrization

In classical statistics, massive models are mathematically intractable and overfit catastrophically. In deep learning, overparametrization actually *simplifies* the optimization landscape, allowing gradient methods to easily find interpolating solutions.

The Linear / "Lazy" Regime

If a network is massively overparametrized, its weights θ barely need to move from their random initialization θ_0 to perfectly fit the data.
We can approximate the network via its first-order Taylor expansion:

$$f(x;\theta) \approx f(x;\theta_0) + \nabla_\theta f(x;\theta_0)^\top (\theta - \theta_0)$$

The network effectively behaves as a linear model in a massively high-dimensional feature space.

Implicit Regularization

Because the model is linear with respect to its parameters, gradient descent behaves predictably: it converges exactly to the **minimum l_2-norm interpolator**.
Out of all the infinite ways to perfectly memorize the training data, gradient descent implicitly chooses the one that keeps the parameters as close to initialization as possible.

Benign Overfitting: The Spiky Decomposition

The Statistical Viewpoint (Bartlett et al., 2021)

For minimum-norm interpolants in high dimensions, the prediction rule mathematically decomposes into two distinct functions: $\hat{f} = \hat{f}_0 + \Delta$.

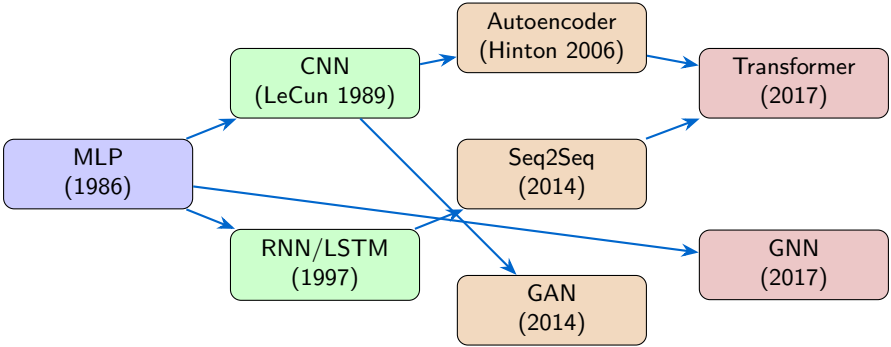
1. The Prediction Component ($\hat{f}_0$)

- **What it is:** A simple, smooth function (often equivalent to low-degree polynomials).
- **Where it lives:** It aligns with the **high-variance subspace** of the data distribution.
- **The Effect:** Because this subspace is low-dimensional, classical uniform convergence holds. It successfully learns the true underlying signal.

2. The Overfitting Component (Δ)

- **What it is:** A highly complex, "spiky" function that perfectly memorizes the training noise.
- **Where it lives:** It is relegated to the orthogonal, the "rich tail".
- **The Effect:** Because the network has millions of parameters, this noise is smeared across infinite dimensions[cite: 4338, 4376]. The spikes are highly localized at the training points and vanish everywhere else.

Evolution of Neural Network Architectures



CNN (ConvNet)

Weight sharing via convolution kernels; translation equivariance. Dominates vision tasks (AlexNet 2012, ResNet 2015).

Transformer

Self-attention replaces recurrence; parallelizable. Powers BERT, GPT, ViT. Attention: $\mathrm{softmax}(QK^\top/\sqrt{d})V$.

GNN

Message passing on graphs: $h_v^{(l+1)} = \sigma\left(Wh_v^{(l)} + \sum_{u\in\mathcal{N}(v)} W' h_u^{(l)}\right)$. Social networks, molecules.

Autoencoders: Definition and Objective

Autoencoder

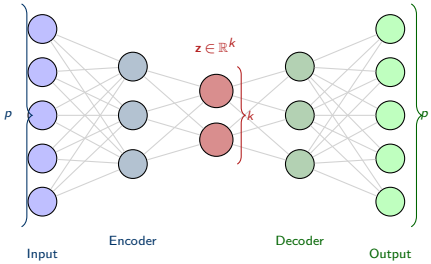
An autoencoder is a neural network trained to *reconstruct its input* through a bottleneck latent representation:

$$\text{Encoder:} \quad \mathbf{z} = e_\phi(\mathbf{x}), \quad \mathbf{z} \in \mathbb{R}^k$$
$$\text{Decoder:} \quad \hat{\mathbf{x}} = d_\psi(\mathbf{z}), \quad \hat{\mathbf{x}} \in \mathbb{R}^p$$

Training objective:

$$\min_{\phi,\psi} \frac{1}{n} \sum_{i=1}^{n} \|\mathbf{x}_i - d_\psi(e_\phi(\mathbf{x}_i))\|^2$$

Bottleneck $k \ll p$ forces learning of a **compressed representation**.



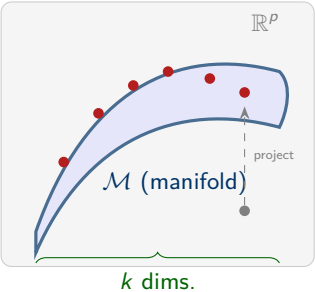
The Manifold Hypothesis and Latent Space

Manifold Hypothesis

Real-world high-dimensional data (images, text, audio) concentrate near a low-dimensional smooth *manifold* $\mathcal{M}$ embedded in $\mathbb{R}^p$:

$$\mathbf{x} \approx f^\star(z), \quad z \in \mathcal{M} \subseteq \mathbb{R}^k, \quad k \ll p$$

The **intrinsic dimension** k captures the true degrees of freedom.



Examples:

- Face images ($p = 10^6$): intrinsic dim $\sim$ pose, expression, illumination ($k \sim 50$)
- Handwritten digits ($p = 784$): $k \sim 10$–20
- Natural language: meaning lives in a much lower-dim semantic space

Key insight: Reconstruction error is small iff the model has learned to represent the data manifold.

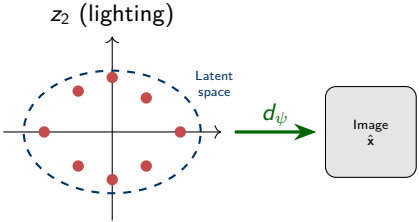
Latent Variables and Disentanglement

Latent variable model

We posit an underlying generative process:

$$z \sim p(z), \quad z \in \mathbb{R}^k$$
$$\mathbf{x} = g^\star(z) + \varepsilon$$

Each coordinate z_j ideally corresponds to a **semantic factor**: rotation, scale, color, etc.



Disentanglement

A *disentangled* representation has:

$$\frac{\partial \hat{x}_i}{\partial z_j} \approx 0 \quad \text{for most } i, j$$

Changing z_j affects only one semantic attribute.

Interpolation in latent space

If the encoder is smooth, **linear interpolation** $z = \alpha z_1 + (1-\alpha) z_2$ in latent space yields semantically meaningful transitions in $\hat{\mathbf{x}}$.

Transformers: The Attention Mechanism

Scaled Dot-Product Attention

Given input $X \in \mathbb{R}^{n \times d}$, project it using learned weights W^Q, W^K, W^V:

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V$$

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

The Database Analogy

- **Query (Q):** "What am I looking for?" (Your search term)
- **Key (K):** "What information do I hold?" (The item's metadata/tags)
- **Value (V):** "What is my actual content?"

Why scale by $\sqrt{d_k}$?

If the dimension d_k is large, the dot products in $QK^\top$ can grow to massive magnitudes. This pushes the Softmax function into its flat tails, causing **vanishing gradients**. Dividing by $\sqrt{d_k}$ normalizes the variance back to 1.

Multi-Head Attention (MHA)

Run h attention heads in parallel, then concatenate:

$$\text{MHA} = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\, W^O$$

Intuition: If you only have one head, it might spend all its capacity looking for "grammar" matches. MHA allows the network to

Self-Attention: A Geometric Perspective

The Learned Metric Space

Why project to Q and K? The term $QK^\top$ computes the inner products in a warped subspace. It acts as a dynamic, data-dependent adjacency matrix A, learning a custom metric where "distance" equals semantic similarity.

The Convex Hull

Because the Softmax function outputs strict probabilities ($\sum p_i = 1$), the attention operation acts as a set of *barycentric coordinates*.

- The output is strictly a **convex combination** of the Value vectors V.
- Every point is pulled toward the geometric center of mass of its semantic neighbors.

Attention on Manifolds (e.g., U-Nets)

In architectures like Diffusion Models, attention acts as a non-local smoothing operator:

- **CNN limits:** Convolutions only operate on the physical 2D grid. They cannot easily connect distant regions.
- **The Semantic Manifold:** Distant image patches (e.g., repeating textures) may be physically far apart but sit adjacent on the underlying data manifold.
- **Manifold Smoothing:** Attention ignores the 2D grid and computes distances along this manifold, identifying disjoint semantic clusters and averaging them to perform global denoising.

PCA as a Linear Autoencoder

PCA recap

Find orthonormal matrix $U \in \mathbb{R}^{p \times k}$ minimizing reconstruction error:

$$\min_{U:\, U^\top U = I_k} \frac{1}{n} \sum_{i=1}^{n} \left\| \mathbf{x}_i - UU^\top \mathbf{x}_i \right\|^2$$

Solution: $U = [v_1, \ldots, v_k]$ (top-k eigenvectors of $\hat{\Sigma}$).

Theorem 3 (Baldi & Hornik 1989)

A linear autoencoder with encoder $W_e \in \mathbb{R}^{k \times p}$ and decoder $W_d \in \mathbb{R}^{p \times k}$ minimizing $\frac{1}{n} \sum \|\mathbf{x} - W_d W_e \mathbf{x}\|^2$:

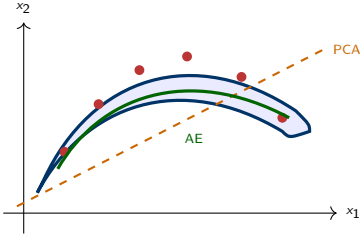
- *Has global minimum equal to PCA reconstruction error*
- *The column space of W_d spans the top-k PCA subspace*
- *Individual columns $\neq$ principal components (not unique, not orthonormal)*

Equivalence: Linear AE $\equiv$ PCA subspace.
Difference: PCA gives a canonical (orthonormal) basis; linear AE does not.

Nonlinear Autoencoders: Beyond PCA

Key difference: curved manifolds

PCA minimizes **linear** reconstruction error — projects onto a hyperplane.
A nonlinear AE can follow **curved** data manifolds.



Distance distortion

Both methods distort distances. PCA distorts *off-manifold* components. AE distorts based on curvature and architecture. For faithful geometry: use **Isomap** or **UMAP**.